

NETWORK SECURITY LABORATORY REPORT

GNS3 + IPFire + Snort 3 Passive IDS Implementation

Laboratory Environment: Kali Linux, IPFire, Metasploitable 2, Ubuntu Secure/Admin, Snort Logger
Date of test activity: 19 September 2026

Purpose	Demonstrate passive network intrusion detection and structured alert logging
Primary IDS	Snort 3.12.2.0 using AFPacket passive capture
Primary evidence	LAB_MASTER_2026-09-19.json (1,358 records)
Topology focus	Kali → IPFire → GREEN segment / hub → protected lab hosts

1. Executive Summary

This project implemented a small, isolated network-security laboratory in GNS3/VMware using IPFire as the firewall/router, Metasploitable 2 as the deliberately vulnerable target, Ubuntu as a relatively secure/admin endpoint, and Snort 3 as a passive network IDS and log collector. The Snort sensor was connected to a hub so that it could observe traffic traversing the protected GREEN network without being placed inline.

The final Snort evidence file contains 1,358 JSON records. The records span approximately 09/19-11:38:28.750677 to 09/19-11:52:30.489783. All 1,358 records are TCP alerts generated by the laboratory rule 1:1000001:1. The dataset includes high-volume TCP reconnaissance against 192.168.10.100 and specific activity associated with TCP/21 and TCP/6200 during the VSFTPD 2.3.4 test.

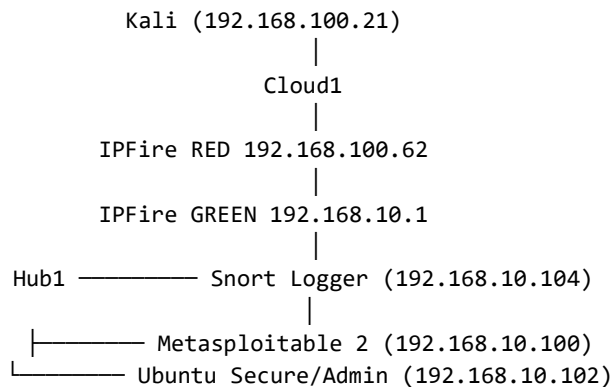
The exercise therefore demonstrated the complete operational path: traffic generation from Kali, firewall traversal through IPFire, observation at the passive Snort sensor, rule matching, and persistent JSON alert storage.

2. Objectives

- Build a segmented network-security laboratory with a firewall, vulnerable host, secure/admin host, and passive IDS.
- Route attack/test traffic from Kali through IPFire into the protected GREEN network.
- Provide Snort visibility by connecting the sensor to a hub rather than placing it inline.
- Validate Snort 3 installation, AFPacket live capture, custom rule loading, and JSON alert logging.
- Generate controlled reconnaissance and application-security test activity and preserve the resulting IDS evidence.

3. Laboratory Architecture

The final logical topology used during testing was:



Component	Address / Interface	Role
Kali Linux	192.168.100.21	Attacker / test generator
IPFire RED	192.168.100.62	External-side firewall interface
IPFire GREEN	192.168.10.1	Protected LAN gateway
Metasploitable 2	192.168.10.100	Deliberately vulnerable target
Ubuntu Secure/Admin	192.168.10.102	Relatively secure endpoint / normal-traffic source
Snort Logger	192.168.10.104 / ens3	Passive IDS and JSON log collector

3.1 Addressing and Segmentation

The physical LAN used 192.168.100.0/24. IPFire separated this network from the protected GREEN network, 192.168.10.0/24. The GREEN DHCP range was configured for 192.168.10.100–192.168.10.200. The VMware VMnet2 segment was used to bridge the existing VMware endpoints into the GNS3 topology.

4. Snort 3 Implementation

Snort 3.12.2.0 was built from source on Ubuntu Server. LibDAQ 3.0.27 was built and installed under /usr/local/lib/daq_s3, and the Snort build was configured against the installed LibDAQ and OpenSSL development libraries.

Validation item	Result
Snort version	3.12.2.0
LibDAQ	3.0.27
Live capture DAQ	AFPacket
Interface	ens3
Capture mode	Passive
Configuration test	Successful; 0 warnings
JSON alert output	/var/log/snort/alert_json.txt
Archived master log	LAB_MASTER_2026-09-19.json

A custom laboratory rule was used to verify that traffic observed by Snort could be matched and written to the JSON logger. The rule was intentionally simple and served as a proof-of-concept detector during the test campaign.

5. Test Campaign

Test	Purpose	Observed result
Network reconnaissance	Generate broad TCP SYN reconnaissance against the protected segment / vulnerable host	Large volume of Snort TCP records; reconnaissance traffic preserved
Metasploitable service enumeration	Identify exposed services on the vulnerable VM	Services including SSH, Telnet, SMTP, DNS, HTTP, SMB, MySQL and PostgreSQL were observed during testing
Nikto web reconnaissance	Generate HTTP-oriented security test traffic	HTTP activity recorded by Snort
UDP reconnaissance	Generate non-TCP reconnaissance traffic	Performed as a test activity; not expected to match the TCP-only custom alert rule
VSFTPD 2.3.4 laboratory test	Test known vulnerable FTP service on Metasploitable	Snort recorded traffic to TCP/21 and TCP/6200
Normal/internal traffic	Generate benign connectivity from Ubuntu Secure/Admin	ICMP/HTTPS activity was performed; not all benign traffic matched the TCP SYN rule

5.1 VSFTPD Evidence

The master Snort log contains 2 records to TCP/21 and 2 records to TCP/6200 during the VSFTPD test window. These records originated from Kali 192.168.100.21 and targeted Metasploitable 192.168.10.100. The presence of TCP/6200 is consistent with the network activity expected from the laboratory VSFTPD 2.3.4 backdoor test; the log should be described as captured exploit-related traffic rather than as proof of a dedicated Snort exploit signature.

6. Evidence Analysis

Indicator	Value
Total Snort records	1,358

Protocol distribution	TCP: 1,358
Rule identifier	1:1000001:1
Primary destination	192.168.10.100 (1,357 records)
Other destination	192.168.10.1 (1 record)
Unique destination ports	1,001
TCP/80 records	112
TCP/21 records	2
TCP/6200 records	2
Secure-host source records	1 record from 192.168.10.102

The first Snort records occur at 11:38:28 and show Kali 192.168.100.21 connecting to a sequence of ports on 192.168.10.100. This is characteristic of the broad TCP reconnaissance test and is preserved as structured JSON evidence.

At approximately 11:45:14, the log records Kali traffic to Metasploitable on TCP/6200 and TCP/21. Later entries include repeated HTTP/TCP/80 activity and a final record from Ubuntu Secure/Admin to IPFire HTTPS management on TCP/444. These entries demonstrate that the same master file contains multiple phases of the laboratory exercise.

7. Logging and Evidence Management

Snort was first validated in passive mode, then executed with the alert_json logger enabled. The live alert file was /var/log/snort/alert_json.txt. After testing, the accumulated dataset was preserved as /var/log/snort/LAB_MASTER_2026-09-19.json. The uploaded copy contains 1,358 lines/records and was used as the primary evidence source for this report.

An IPFire firewall log was also collected during the laboratory work. However, the supplied IPFire text file uses timestamps that do not align directly with the 11:38–11:52 Snort timestamps, so this report treats Snort as the primary correlated IDS evidence rather than claiming a precise one-to-one firewall/IDS time correlation.

8. Security and Operational Considerations

- The environment is a controlled laboratory containing an intentionally vulnerable host. The traffic-generation activities should remain isolated from production networks.
- The current laboratory TCP SYN rule is suitable for demonstrating packet visibility and alert logging, but it is not a production-quality intrusion-detection policy because it can generate significant noise.
- A realistic next phase would retain broad passive packet visibility while replacing the proof-of-concept rule with a curated set of Snort signatures, thresholds, suppressions, and protocol-specific detections.
- For repeatable experiments, the lab should archive the Snort JSON file together with the exact test commands, timestamps, IP plan, and rule version used for the run.

9. Conclusion

The laboratory successfully demonstrated a passive network IDS architecture using GNS3, IPFire, Metasploitable 2, Ubuntu Secure/Admin, and Snort 3. Traffic generated by Kali traversed the firewall into the protected network and was visible to the Snort sensor through the hub. Snort successfully loaded the custom laboratory rule, captured traffic through AFPacket in passive mode, generated alerts, and persisted them in JSON format.

The resulting master evidence set contains 1,358 structured records and includes reconnaissance activity, HTTP activity, VSFTPD-related traffic, and at least one later benign HTTPS management event. This provides a sufficient baseline for the laboratory demonstration and for subsequent analysis, screenshots, presentation material, or a Git-based project archive.

Appendix A - Key Log Examples

Reconnaissance

```
{"timestamp": "09/19-11:38:28.750677", "pkt_num": 25656, "proto": "TCP", "pkt_gen": "raw", "pkt_len": 44, "dir": "C2S", "src_ap": "192.168.100.21:61858", "dst_ap": "192.168.10.100:256", "rule": "1:1000001:1", "action": "allow"}
```

VSFTPD-related TCP/6200

```
{"timestamp": "09/19-11:45:14.555743", "pkt_num": 35861, "proto": "TCP", "pkt_gen": "raw", "pkt_len": 60, "dir": "C2S", "src_ap": "192.168.100.21:36697", "dst_ap": "192.168.10.100:6200", "rule": "1:1000001:1", "action": "allow"}
```

FTP TCP/21

```
{"timestamp": "09/19-11:38:28.756713", "pkt_num": 25660, "proto": "TCP", "pkt_gen": "raw", "pkt_len": 44, "dir": "C2S", "src_ap": "192.168.100.21:61858", "dst_ap": "192.168.10.100:21", "rule": "1:1000001:1", "action": "allow"}
```

Later HTTPS management

```
{"timestamp": "09/19-11:52:30.489783", "pkt_num": 60671, "proto": "TCP", "pkt_gen": "raw", "pkt_len": 60, "dir": "C2S", "src_ap": "192.168.10.102:52060", "dst_ap": "192.168.10.1:444", "rule": "1:1000001:1", "action": "allow"}
```

Evidence sources: Uploaded LAB_MASTER_2026-09-19.json and firewalllog.txt, plus the documented laboratory configuration and test history from the project session. No external source was used to fill missing lab-specific facts.

Correlated Security Findings and Final Analysis

1. Correlation of the Two Logging Layers

The laboratory produced two complementary logging layers: Snort recorded packet-level IDS observations on the internal monitoring segment, while IPFire recorded firewall forwarding and input-policy decisions. The two sources should therefore be interpreted together rather than treated as duplicate logs. Snort demonstrates that reconnaissance traffic was visible on the monitored network; IPFire demonstrates how the firewall subsequently handled traffic according to its configured policy.

2. Reconnaissance Activity — 11:38 to 11:52

During the first observed activity window, Kali (192.168.100.21) generated a large TCP reconnaissance scan against Metasploitable (192.168.10.100). The Snort JSON dataset contains the individual TCP SYN packets from Kali to the target across a broad sequential port range. The records use rule 1:1000001:1 and show action 'allow'. This is consistent with the current laboratory rule, which was designed to alert on TCP SYN activity rather than aggregate packets into a dedicated port-scan classification.

3. Important Snort Finding

The scan was successfully visible and logged by Snort, but the current custom rule did not classify the sequence as a single 'port scan' event. Therefore, the correct conclusion is not that Snort failed to see the scan. Rather, the current rule set demonstrated packet-level visibility without higher-level scan aggregation or classification. This distinction is important when evaluating IDS functionality.

4. IPFire Firewall Activity — Later Session

The IPFire firewall log shows a later activity period involving the same attacker address, 192.168.100.21. The records include FORWARDFW activity toward the GREEN network, as well as DROP_FORWARD and DROP_INPUT events for traffic that did not satisfy the applicable firewall policy. The firewall therefore provided a separate enforcement layer: some traffic was permitted according to the configured rule set, while other traffic was explicitly dropped.

5. Internal Host Discovery and Gateway Probing

The later firewall evidence includes ICMP activity involving internal addresses including 192.168.10.100, 192.168.10.102 and 192.168.10.104, with dropped forwarding events visible in the firewall log. It also includes repeated connection attempts from the attacker toward the IPFire WebGUI address 192.168.10.1 on TCP/444, recorded as DROP_INPUT. These events demonstrate the value of retaining firewall enforcement logs alongside IDS observations.

6. Metasploitable / FTP Evidence

Metasploitable at 192.168.10.100 was confirmed as a reachable vulnerable laboratory host with numerous exposed services. The Snort evidence includes traffic associated with FTP and the VSFTPD laboratory activity, including TCP/21 and subsequent TCP/6200 traffic. This provides evidence that the test was performed against an intentionally vulnerable service rather than only against an inactive scan target.

7. Background Traffic and Noise

The firewall logs also contain traffic that should not automatically be classified as malicious. Examples include directed/broadcast LAN traffic associated with hosts such as 192.168.100.11, 192.168.100.27 and 192.168.100.8, including NetBIOS and local-discovery traffic. There are also DROP_CTINVALID entries associated with routine outbound Ubuntu/Canonical update traffic. These observations demonstrate why security logs require contextual analysis rather than treating every dropped packet as an attack.

8. Final Security Architecture Finding

The laboratory successfully demonstrated defense-in-depth using separate network-security functions. IPFire provided routing, segmentation and firewall enforcement between the RED and GREEN networks. The hub-based monitoring path allowed the passive Snort sensor to observe internal traffic without becoming an inline dependency. Snort generated structured JSON records that preserved source, destination, protocol, ports, timestamps and rule identifiers. Together, these components provide a practical foundation for network monitoring and incident investigation.

9. Evidence Summary

The final Snort master dataset contains 1,358 JSON records from the laboratory session. The evidence demonstrates TCP reconnaissance from Kali, internal traffic toward Metasploitable, VSFTPD-related activity, and subsequent traffic involving the secure Ubuntu and monitoring environment. IPFire provides a second, independent evidence layer showing forwarding and blocking decisions. The two datasets are complementary even where their timestamps or retention windows do not align perfectly.

10. Limitations and Recommended Future Improvements

The current laboratory rule is intentionally simple and should not be presented as a complete port-scan detection capability. A production-oriented Snort deployment would use a broader, maintained rule set and appropriate thresholding/detection mechanisms for reconnaissance, service probing and other attack classes. Future work could also forward IPFire, endpoint and Snort logs to a centralized SIEM or log-management platform, synchronize timestamps using NTP, and retain packet captures or additional flow metadata for deeper forensic correlation.

Final Conclusion

The completed laboratory demonstrates a functioning segmented network-security environment in which Kali generated controlled reconnaissance and service-testing traffic, IPFire enforced network policy, and passive Snort observed and recorded internal traffic in structured JSON format. The most significant analytical result is that the current Snort rule captured the scan packets but did not aggregate them into a dedicated port-scan alert, while IPFire independently recorded its enforcement decisions. This provides a realistic and technically defensible basis for demonstrating network monitoring, firewall enforcement, IDS logging, and security-event correlation.